

## **Лекция 1.**

# **Дисковая подсистема Linux-сервера: LVM и программный RAID**

## Цель лекции

Понять слои хранения данных в Linux-сервере

Собрать простую цепочку RAID1 → LVM → ext4 →  
/srv/apps

Создавать PV, VG и LV средствами LVM

Создавать и проверять программный RAID1 через  
mdadm

Настраивать постоянное монтирование по UUID в  
/etc/fstab

Диагностировать заполнение диска, inode, RAID и  
ошибки монтирования

## Учебные вопросы

1. Архитектура хранения: диск, раздел, блочное устройство, файловая система и точка монтирования
2. LVM: PV, VG, LV; создание и расширение томов
3. LVM Snapshot — обзор возможностей и ограничений
4. RAID1 и обзор RAID5/RAID10
5. Проверка дисков и файловых систем: mdadm, SMART, df, du, inode
6. Монтирование по UUID и /etc/fstab
7. Базовая диагностика заполнения диска, RAID и ошибок монтирования

## Главный практический результат

Вся лекция постепенно собирает одну архитектуру хранения

`/dev/sdb` и `/dev/sdc` становятся участниками RAID1

RAID-массив `/dev/md0` становится Physical Volume для LVM

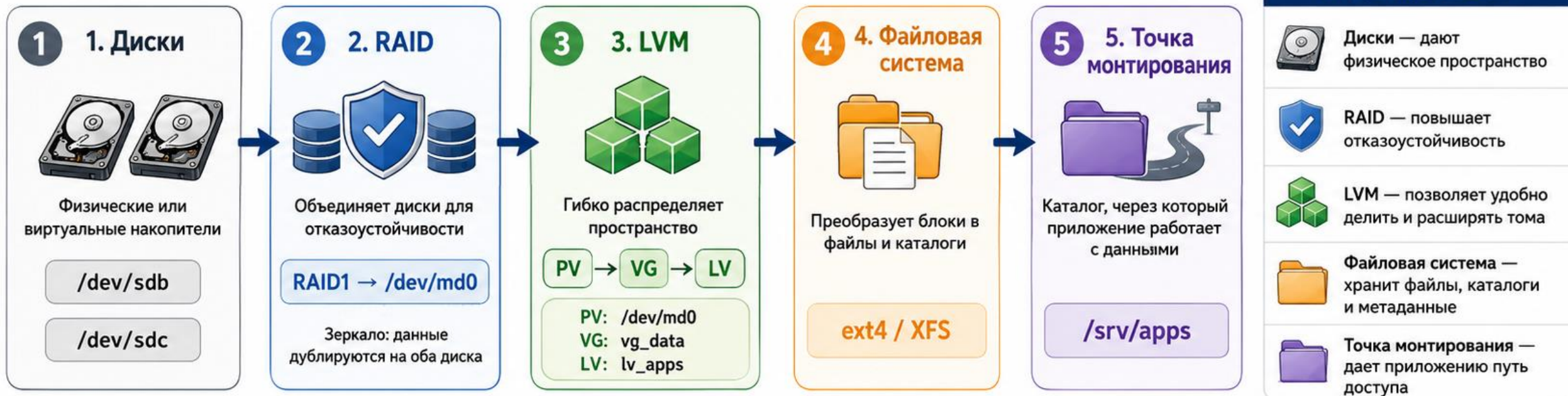
Volume Group `vg_data` содержит пространство RAID-массива

Logical Volume `lv_apps` получает файловую систему `ext4`

Каталог `/srv/apps` становится постоянной точкой  
МОНТИРОВАНИЯ

# ДИСКИ, RAID, LVM, ФАЙЛОВАЯ СИСТЕМА И ТОЧКА МОНТИРОВАНИЯ

Как данные проходят путь от физических дисков до каталога приложения



## ПРИМЕР ЦЕПОЧКИ



## ПОЛЕЗНЫЕ КОМАНДЫ

lsblk +f



диски, разделы,  
LVM, точки монтирования

cat /proc/mdstat



состояние RAID

pvs | vgs | lvs



состояние LVM

blkid



UUID и тип ФС

findmnt /srv/apps



что смонтировано  
в каталог

df -hT /srv/apps



тип ФС и свободное место



### RAID ≠ backup

RAID защищает от отказа диска, но не от удаления данных, вирусов и ошибок пользователей.



### LVM часто работает поверх RAID

RAID дает отказоустойчивость, LVM добавляет гибкость управления и масштабирования.



### Перед запуском приложения всегда проверяют mount point

Неправильная точка монтирования может привести к потере данных или просто.

## 1. Хранение данных в Linux строится слоями

Disk — физическое или виртуальное устройство хранения

Partition — часть диска

Block Device — устройство, с которым ОС работает блоками

Filesystem — структура файлов и каталогов

Mount Point — каталог, куда подключена файловая система

Ошибка на нижнем уровне влияет на все уровни выше

## Диск и файловая система — разные сущности

Диск может существовать без разделов

Раздел может существовать без файловой системы

Файловая система может быть создана на разделе,  
RAID или LV

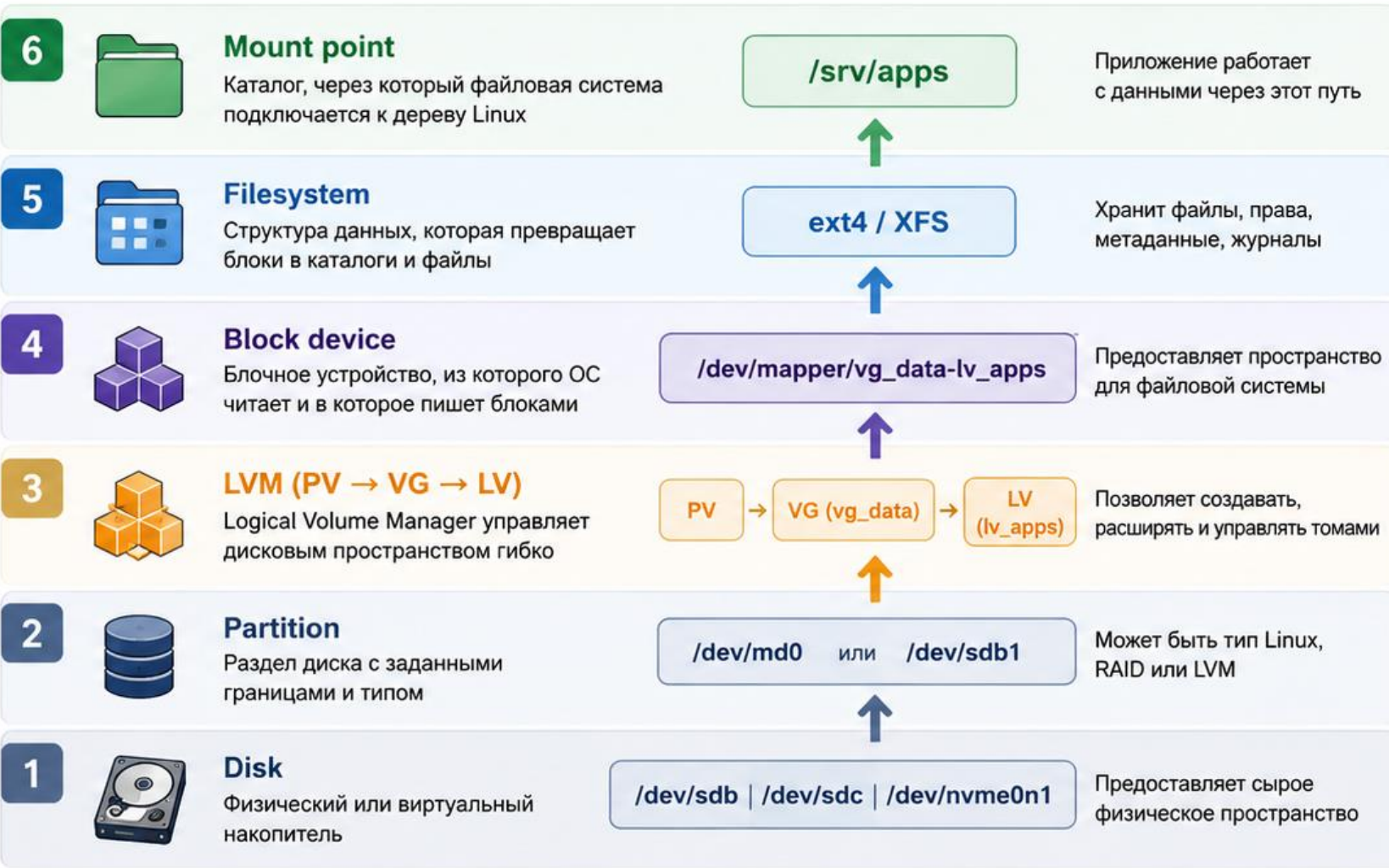
Каталог может существовать без смонтированной  
файловой системы

Администратор должен понимать, на каком уровне он  
выполняет команду



# СЛОИ ХРАНЕНИЯ ДАННЫХ В LINUX

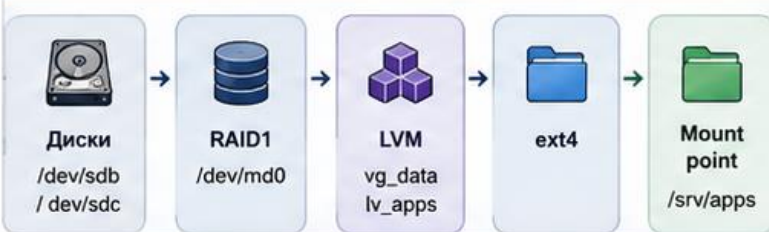
Путь от физического диска до каталога приложения



## КАК ЧИТАТЬ СХЕМУ

- ↑ Данные идут **СВЕРХУ ВНИЗ** при записи  
Приложение → файлы → блоки → устройство → диск
- ↓ Данные идут **СНИЗУ ВВЕРХ** при чтении  
Диск → устройство → блоки → файлы → приложение
- 💡 Каждый слой имеет свою задачу  
Путаница между устройством, ФС и точкой монтирования — частая причина ошибок

## ПРИМЕР ЦЕПОЧКИ ДЛЯ /srv/apps



## ПОЛЕЗНЫЕ КОМАНДЫ

- |                          |                                                   |
|--------------------------|---------------------------------------------------|
| <b>lsblk -f</b>          | Показать диски, разделы, LVM и точки монтирования |
| <b>blkid</b>             | Показать UUID и тип файловой системы              |
| <b>fdisk -l</b>          | Показать таблицу разделов и размеры дисков        |
| <b>findmnt /srv/apps</b> | Показать, что смонтировано в каталоге             |
| <b>df -hT /srv/apps</b>  | Показать тип ФС и свободное место                 |



### ВАЖНО ПОМНИТЬ



Проверяйте точку монтирования после настройки хранения



Ошибки часто связаны с тем, что данные пишутся не туда, куда ожидается



RAID защищает от отказа дисков, LVM даёт гибкость, а файловая система хранит данные



Планируйте, проверяйте и регулярно контролируйте состояние системы хранения



## Точка монтирования определяет, куда идут данные

/srv/apps смонтирован → данные приложения идут в отдельную файловую систему

/srv/apps не смонтирован → приложение может писать в корневую файловую систему

Это может быстро заполнить /

Перед запуском сервиса нужно проверить mount point

Команда: `findmnt /srv/apps`

Команда: `df -hT /srv/apps`

# RAID, LVM, файловая система и mount point решают разные задачи

RAID → отказоустойчивость дисков

LVM → гибкое распределение пространства

Filesystem → файлы и каталоги

Mount Point → путь приложения

Главная цепочка: Disks → RAID → LVM → Filesystem →

Mount Point

RAID и LVM не заменяют файловую систему

## Перед изменениями сначала смотрим карту хранения

Команда: `lsblk -f`

Назначение: увидеть диски, разделы, файловые системы, UUID и точки монтирования

Команда: `sudo fdisk -l`

Назначение: увидеть диски, размеры и таблицы разделов

Команда: `sudo blkid`

Назначение: увидеть UUID и типы файловых систем

Нельзя запускать `pvcreate`, `mkfs` или `mdadm --create`, пока диск не проверен

## Предупреждение перед разрушительными командами

`sudo pvcreate` подготавливает устройство для LVM

`sudo mkfs.ext4` создает новую файловую систему

`sudo mdadm --create` создает RAID-массив

Эти команды могут уничтожить старые данные

Перед вводом команды нужно убедиться, что выбран именно учебный диск

В нашей лаборатории используются `/dev/sdb` и `/dev/sdc`

## 2. LVM добавляет гибкость поверх устройства

LVM (Logical Volume Manager) — менеджер логических томов Linux

LVM позволяет создавать логические диски поверх одного или нескольких устройств

Размер LV можно расширять без пересоздания всей структуры

В лаборатории LVM будет работать поверх RAID1-массива `/dev/md0`

LVM не заменяет RAID и не является backup



## PV, VG и LV — три базовых уровня LVM

PV (Physical Volume) — устройство, подготовленное для LVM

VG (Volume Group) — общий пул пространства из одного или нескольких PV

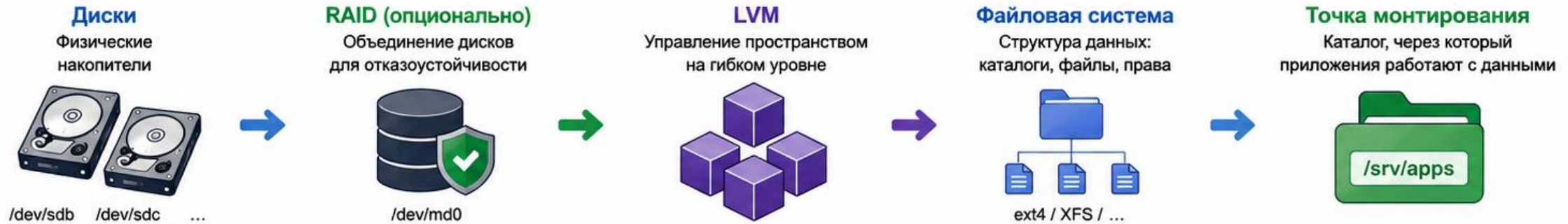
LV (Logical Volume) — логический диск, выделенный из VG

Файловая система создается уже на LV

В нашем сценарии: /dev/md0 → vg\_data → lv\_apps

# PHYSICAL VOLUME, VOLUME GROUP И LOGICAL VOLUME

Как LVM организует гибкое и отказоустойчивое хранение в Linux



## LVM: ТРИ ОСНОВНЫХ ЭЛЕМЕНТА

### PV – Physical Volume Физический том



PV

- Представляет собой диск или раздел
- Помечается для использования LVM
- Содержит метаданные LVM
- Может быть частью RAID-устройства

`/dev/sdb1` `/dev/md0` `/dev/nvme0n1p1`

### VG – Volume Group Группа томов



vg\_data

- Объединяет несколько PV в единое пул-пространство
- Пространство VG можно распределять между LV
- Легко расширяется добавлением новых PV
- Не имеет файловой системы

### LV – Logical Volume Логический том



lv\_apps

- Выделяется из свободного пространства VG
- Может быть любого размера
- Можно расширять или уменьшать без потери данных\*
- На LV создаётся файловая система

## ПОЛЕЗНЫЕ КОМАНДЫ



### PV – Physical Volume

```
pvcreeate /dev/sdb1
pvdisplay
pvs
```



### VG – Volume Group

```
vgcreate vg_data /dev/sdb1 /dev/md0
vgdisplay
vgs
```



### LV – Logical Volume

```
lvcreate -L 50G -n lv_apps vg_data
lvdisplay
lvs
```



### Файловая система

```
mkfs.ext4 /dev/vg_data/lv_apps
tune2fs -L APPS /dev/vg_data/lv_apps
```



### Монтирование

```
mount /dev/vg_data/lv_apps /srv/apps
df -hT /srv/apps
```

## ПРЕИМУЩЕСТВА LVM



- Гибкое управление пространством
- Легкое расширение томов «на лету»
- Снимки (snapshots) для резервного копирования
- Перенос томов между серверами
- Абстракция от физической структуры дисков

## ПРИМЕР ЦЕПОЧКИ ХРАНЕНИЯ ДАННЫХ



## ВАЖНО ПОМНИТЬ



- Удаление PV/VG/LV может привести к потере данных
- Всегда проверяйте устройства перед командами
- Держите резервные копии важных данных
- Используйте UUID при монтировании в `/etc/fstab`

\* Расширение LV и файловой системы: сначала увеличиваем LV (`lvextend`), затем файловую систему (`resize2fs` для `ext4` или `xfs_growfs` для `XFS`).

## Создание LVM поверх RAID1

Команда: `sudo pvcreate /dev/md0`

Назначение: сделать RAID-массив Physical Volume

Команда: `sudo vgcreate vg_data /dev/md0`

Назначение: создать Volume Group `vg_data`

Команда: `sudo lvcreate -n lv_apps -L 10G vg_data`

Назначение: создать Logical Volume `lv_apps` размером 10G

## Проверка LVM после создания

Команда: `sudo pvs`

Проверяет Physical Volume и его принадлежность к VG

Команда: `sudo vgs`

Проверяет размер Volume Group и свободное место

Команда: `sudo lvs`

Проверяет Logical Volume, размер и путь устройства

## Создаем ext4 и монтируем /srv/apps

Команда: `sudo mkfs.ext4 /dev/vg_data/lv_apps`

Назначение: создать файловую систему ext4

Команда: `sudo mkdir -p /srv/apps`

Назначение: подготовить точку монтирования

Команда: `sudo mount /dev/vg_data/lv_apps /srv/apps`

Назначение: подключить LV к каталогу приложения

**Проверяем, что /srv/apps действительно работает отдельно**

Команда: lsblk -f

Ожидаемо: lv\_apps имеет файловую систему ext4

Команда: findmnt /srv/apps

Ожидаемо: /srv/apps смонтирован с  
/dev/mapper/vg\_data-lv\_apps

Команда: df -hT /srv/apps

Ожидаемо: отдельная файловая система ext4 для  
/srv/apps



## Расширение LVM выполняется на двух уровнях

Первый уровень: увеличить Logical Volume

Второй уровень: увеличить файловую систему внутри LV

Команда: `sudo lvextend -L +5G /dev/vg_data/lv_apps`

Команда: `sudo resize2fs /dev/vg_data/lv_apps`

Проверка: `df -h /srv/apps`

Если увеличить только LV, файловая система может не увидеть новое место

## Опция -r упрощает расширение

Команда: `sudo lvextend -r -L +5G /dev/vg_data/lv_apps`  
-r автоматически запускает расширение файловой системы

Это удобно в практике администратора

Но студент должен понимать два уровня: LV и filesystem

Перед расширением нужно проверить свободное место: `sudo vgs`

После расширения нужно проверить результат: `df -h /srv/apps`

## Уменьшение томов — только обзорно

Расширение — базовая и часто безопасная операция

Уменьшение — более рискованная процедура

ext4 уменьшается только по отдельной процедуре

XFS штатно не уменьшается

Перед уменьшением нужен backup

В базовой лаборатории уменьшение томов не выполняем

## XFS — обзорно, без отдельной практики

ext4 используется как основная файловая система  
лаборатории

XFS распространена на Linux-серверах

XFS хорошо расширяется

XFS штатно не уменьшается

Дополнительный пример: `sudo xfs_growfs /srv/apps`

Команду `xfs_growfs` применяют к смонтированной XFS-  
файловой системе

### 3. LVM Snapshot фиксирует состояние LV

LVM Snapshot — снимок состояния Logical Volume на момент создания

Команда: `sudo lvcreate -s -n snap_apps -L 2G /dev/vg_data/lv_apps`

`snap_apps` — имя snapshot

`2G` — место для изменений после снимка

Проверка: `sudo lvs`

Snapshot не является backup

## Snapshot имеет ограничения

Snapshot находится на той же storage-системе

Если хранилище потеряно, snapshot тоже потерян

Snapshot использует отдельное место для изменений

Если snapshot переполнится, он станет бесполезным

Crash-consistent означает состояние как после

внезапного выключения

Application-consistent требует подготовки приложения  
и является дополнительной темой



## Удаление snapshot после проверки

Snapshot не нужно держать бесконечно

Старый snapshot расходует место и усложняет сопровождение

Команда: `sudo lvremove /dev/vg_data/snap_apps`

Перед удалением убедиться, что снимок больше не нужен

Backup и snapshot решают разные задачи

Для долговременного хранения нужен отдельный backup

## 4. RAID повышает отказоустойчивость дисков

RAID (Redundant Array of Independent Disks)

объединяет несколько дисков в массив

RAID1 хранит одинаковые данные на двух дисках

Если один диск отказал, второй продолжает содержать данные

RAID помогает пережить отказ диска

RAID не защищает от удаления файла, вируса, ошибки администратора или пожара

RAID не является Backup

## RAID1, RAID5 и RAID10: базовое сравнение

RAID1: минимум 2 диска, полезная емкость около 50%, допускает отказ 1 диска

RAID5: минимум 3 диска, полезная емкость  $N-1$  дисков, допускает отказ 1 диска

RAID10: минимум 4 диска, полезная емкость около 50%

RAID10 допускает отказ дисков в зависимости от зеркальных пар

В лаборатории основной практикой является RAID1  
RAID5 и RAID10 здесь нужны только для сравнения

# RAID1, RAID5 и RAID10

Сравнение уровней RAID по отказоустойчивости, полезной ёмкости и типовым сценариям

## КЛЮЧЕВЫЕ ОТЛИЧИЯ



**RAID1** — самое простое зеркало



**RAID5** — экономит место, но есть штраф записи



**RAID10** — сочетает скорость и отказоустойчивость



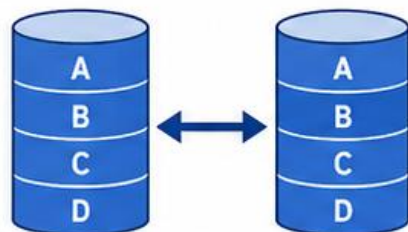
**RAID10** может пережить несколько отказов, если они не в одной зеркальной паре



**RAID5** рискован при долгом rebuild больших дисков

## RAID1

ЗЕРКАЛО (MIRROR)

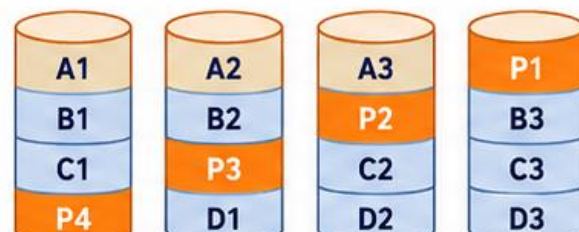


Диск 1      Диск 2

- Минимум: 2 диска
- Полезная ёмкость: 50%
- Отказ: обычно 1 диск
- Сильная сторона: простое зеркало

## RAID5

ЧЕРЕДОВАНИЕ С ПАРИТЕТОМ



Диск 1      Диск 2      Диск 3      Диск 4

- Минимум: 3 диска
- Полезная ёмкость:  $N - 1$  дисков
- Отказ: 1 диск
- Сильная сторона: баланс ёмкости и защиты

## RAID10

ЗЕРКАЛЬНЫЕ ПАРЫ + ЧЕРЕДОВАНИЕ



Диск 1      Диск 2      Диск 3      Диск 4

- Минимум: 4 диска
- Полезная ёмкость: 50%
- Отказ: зависит от пар
- Сильная сторона: высокая производительность

## СРАВНЕНИЕ

| Уровень       | Мин. дисков | Полезная ёмкость | Допустимый отказ    | Типовое применение                              |
|---------------|-------------|------------------|---------------------|-------------------------------------------------|
| <b>RAID1</b>  | 2           | 50%              | 1 диск              | Небольшие серверы, ОС, критичные данные         |
| <b>RAID5</b>  | 3           | $(N-1)/N$        | 1 диск              | Файловые хранилища, где важна ёмкость           |
| <b>RAID10</b> | 4           | 50%              | 1+ при разных парах | Базы данных, виртуализация, нагруженные сервисы |

## ЧТО ВАЖНО ПОМНИТЬ



RAID ≠ backup



Rebuild нагружает диски



Чем больше диски, тем дольше восстановление



Для интенсивной нагрузки чаще выбирают RAID1 или RAID10

## ТИПОВАЯ ПРОВЕРКА В LINUX

```
cat /proc/mdstat
```

состояние массива

```
mdadm --detail /dev/md0
```

подробности RAID

```
lsblk
```

видно диски и md-устройства



**RAID повышает отказоустойчивость, но не заменяет резервное копирование.**

## Создаем программный RAID1 через mdadm

Команда: `sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc`

`/dev/md0` — имя RAID-массива

`--level=1` — RAID1

`--raid-devices=2` — два участника массива

`/dev/sdb` и `/dev/sdc` — учебные диски

Перед командой обязательно проверить диски через `lsblk -f`

## Проверяем RAID1 после создания

Команда: `cat /proc/mdstat`

Назначение: увидеть состояние RAID и синхронизацию

Команда: `sudo mdadm --detail /dev/md0`

Назначение: увидеть участников массива и состояние md0

Команда: `watch cat /proc/mdstat`

Назначение: наблюдать процесс синхронизации в динамике



## Состояние RAID1 читаем просто

[UU] → оба диска работают

[U\_] → один участник отсутствует или неисправен

resync → синхронизация массива

recovery → восстановление после добавления или замены диска

Для Junior достаточно уверенно читать /proc/mdstat и mdadm --detail

Сложный reshape в этой лекции не рассматриваем



## **Дополнительно: автоматическая сборка RAID после reboot**

После создания RAID важно, чтобы система собрала массив после перезагрузки

Команда: `sudo mdadm --detail --scan`

В Debian/Ubuntu конфигурация хранится в `/etc/mdadm/mdadm.conf`

Тема `initramfs` существует, но не является основной для этой лекции

В базовой лаборатории достаточно понять, что RAID должен собираться автоматически

## RAID + LVM работают как разные слои

RAID создает отказоустойчивое блочное устройство  
/dev/md0

LVM использует /dev/md0 как Physical Volume

Команда: `sudo pvcreate /dev/md0`

Команда: `sudo vgcreate vg_data /dev/md0`

RAID отвечает за физическую отказоустойчивость

LVM отвечает за гибкие логические тома

## 5. SMART помогает оценить физический диск

SMART (Self-Monitoring, Analysis and Reporting Technology) хранит показатели состояния диска

Команда: `sudo smartctl -a /dev/sda`

Для команды нужен пакет `smartmontools`

SMART полезен для физического сервера

В VirtualBox SMART может быть недоступен

В этой лекции SMART рассматривается обзорно

## **df, du и inode отвечают на разные вопросы**

Команда: `df -h`

Показывает заполнение файловых систем

Команда: `sudo du -xhd1 /var`

Показывает, какие каталоги занимают место внутри /var

Команда: `df -i`

Показывает использование inode

## **inode могут закончиться раньше гигабайтов**

inode — служебная запись о файле в файловой системе

Если inode = 100%, новые файлы не создаются

При этом df -h может показывать свободные гигабайты

Типичный пример: очень много мелких файлов

Проверка: df -i

Вывод: свободное место и свободные inode нужно проверять отдельно

## Дополнительно: df и du могут расходиться

df показывает занятое место на файловой системе

du суммирует видимые файлы в каталогах

Если файл удален, но открыт процессом, df может показывать место занятым

du этот файл уже не видит

Команда: `sudo lsof +L1`

На больших файловых системах du и find могут создавать заметную I/O-нагрузку

## 6. UUID делает монтирование устойчивым

Имена устройств вроде `/dev/sdb` могут измениться после перезагрузки

UUID — уникальный идентификатор файловой системы

Команда: `sudo blkid /dev/vg_data/lv_apps`

Пример строки `/etc/fstab`: `UUID=<UUID> /srv/apps ext4 defaults 0 2`

UUID надежнее имени диска

После изменения `/etc/fstab` нельзя сразу перезагружать сервер без проверки



## Проверка `fstab` до `reboot` обязательна

Команда: `sudo mount -a`

Назначение: проверить, что все записи `/etc/fstab` монтируются без ошибки

Команда: `findmnt /srv/apps`

Назначение: проверить точку монтирования

Команда: `df -hT /srv/apps`

Назначение: проверить файловую систему и размер

Дополнительно: `findmnt --verify`

## **nofail применяют осторожно**

nofail позволяет системе продолжить загрузку, если том не найден

Это полезно для необязательных томов

Для критического /srv/apps nofail может скрыть проблему

Сервис может стартовать и писать данные не туда

Перед использованием nofail нужно понять, критичен ли том для приложения

## Файлы под mount point могут стать невидимыми

Если в /srv/apps уже были файлы, они лежали в корневой файловой системе

После mount нового LV поверх /srv/apps старые файлы становятся невидимыми

Они не удалены, но скрыты смонтированной файловой системой

Перед первым mount: `sudo ls -la /srv/apps`

После mount: `findmnt /srv/apps`

Это важный кейс при переносе данных приложения

## 7. Диагностика хранения идет по уровням

Сначала проверяем карту устройств: `lsblk`

Затем проверяем mount point: `findmnt`

Потом проверяем место: `df -h`

Потом проверяем inode: `df -i`

Затем ищем крупные каталоги: `du`

Для RAID проверяем `/proc/mdstat` и `mdadm --detail`

# ДИАГНОСТИКА ПРОБЛЕМ ДИСКОВОЙ ПОДСИСТЕМЫ

Как быстро найти и решить проблемы с дисками, RAID, LVM, файловыми системами и монтированием

## 1. СИМПТОМЫ И ВОЗМОЖНЫЕ ПРИЧИНЫ

### Диск заполнен



- Большие файлы и логи
- Копирование данных
- Нет ротации журналов
- Неочищенный кэш

### Не хватает inode



- Много мелких файлов
- Распаковка архивов
- Временные файлы
- Некорректная очистка

### Медленная работа



- Высокая нагрузка I/O
- Фрагментация
- Медленные диски
- Проблемы RAID/LVM

### RAID в состоянии degraded



- Отказ диска
- Кабель/контроллер
- Сбой синхронизации
- Нехватка места для rebuild

### Ошибка монтирования



- Неверный UUID/путь
- Повреждена ФС
- Неправильные права
- Нет зависимостей (LVM/RAID)

### LVM том не активен



- VG не найден
- PV отсутствует
- Повреждены метаданные
- Нет свободного места

## 2. КОМАНДЫ ДЛЯ ДИАГНОСТИКИ

| Что проверяем       | Команда                                   | Пример                                    | Что покажет                                        |
|---------------------|-------------------------------------------|-------------------------------------------|----------------------------------------------------|
| Диски и разделы     | <code>lsblk -f</code>                     | <code>lsblk -f</code>                     | Дерево устройств, тип ФС, UUID, точки монтирования |
| Использование места | <code>df -hT</code>                       | <code>df -hT</code>                       | Свободное место по файловым системам и их тип      |
| Использование inode | <code>df -i</code>                        | <code>df -i</code>                        | Свободные inode, процент использования             |
| Размер каталогов    | <code>du -sh /*</code>                    | <code>du -sh /*</code>                    | Какие каталоги занимают больше всего               |
| RAID статус         | <code>cat /proc/mdstat</code>             | <code>cat /proc/mdstat</code>             | Состояние массивов: active, degraded, rebuilding   |
| Детали RAID         | <code>sudo mdadm --detail /dev/md0</code> | <code>sudo mdadm --detail /dev/md0</code> | Уровень, диски, состояние, время на rebuild        |
| LVM информация      | <code>pvs; vgs; lvs</code>                | <code>pvs; vgs; lvs</code>                | PV, VG, LV: размеры, свободное место, статусы      |
| Журнал событий      | <code>journalctl -xe</code>               | <code>journalctl -xe</code>               | Ошибки системы, дисков, RAID, LVM, монтирования    |
| SMART дисков        | <code>sudo smartctl -a /dev/sda</code>    | <code>sudo smartctl -a /dev/sda</code>    | Состояние диска, ошибки, температура, прогноз      |
| Проверка ФС         | <code>sudo fsck -n /dev/sda1</code>       | <code>sudo fsck -n /dev/sda1</code>       | Проверка файловой системы (без исправлений)        |

## 3. ТИПИЧНЫЕ ПРОБЛЕМЫ И ЧТО ДЕЛАТЬ

| Проблема            | Как определить                                                                         | Что делать                                                                                                                                                                     |
|---------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Диск заполнен       | <code>df -hT</code> → Use% = 100%<br><code>du -sh /*</code> → найти большой каталог    | <ul style="list-style-type: none"><li>• Очистить /var/log, кэш, временные файлы</li><li>• Удалить ненужные данные</li><li>• Настроить ротацию логов (logrotate)</li></ul>      |
| Не хватает inode    | <code>df -i</code> → IUse% = 100%<br><code>du -ahx --inodes   sort -nr   head</code>   | <ul style="list-style-type: none"><li>• Найти каталоги с большим числом файлов</li><li>• Удалить лишние мелкие файлы</li><li>• Использовать ФС с большим inode ratio</li></ul> |
| RAID degraded       | <code>cat /proc/mdstat</code><br><code>mdadm --detail /dev/md0</code>                  | <ul style="list-style-type: none"><li>• Определить отказавший диск</li><li>• Заменить диск</li><li>• Запустить восстановление (rebuild)</li></ul>                              |
| LVM том недоступен  | <code>pvs / vgs / lvs</code> → том не отображается<br><code>dmesg   grep -i lvm</code> | <ul style="list-style-type: none"><li>• Проверить наличие PV и VG</li><li>• Активировать VG: <code>vgchange -ay</code></li><li>• Восстановить метаданные LVM</li></ul>         |
| Ошибка монтирования | <code>mount -a</code> → ошибка<br><code>dmesg   tail</code>                            | <ul style="list-style-type: none"><li>• Проверить UUID и путь в /etc/fstab</li><li>• Проверить ФС: <code>fsck</code></li><li>• Исправить права и зависимости</li></ul>         |
| Медленная работа    | <code>iostat -xz 1</code><br><code>iotop -oP</code><br><code>vmstat 1</code>           | <ul style="list-style-type: none"><li>• Проверить нагрузку и I/O</li><li>• Найти «тяжёлые» процессы</li><li>• Рассмотреть апгрейд дисков/RAID/LVM</li></ul>                    |

## 4. ОБЩИЙ АЛГОРИТМ ДИАГНОСТИКИ

- 1 Проверить симптомы и получить жалобы пользователя
- 2 Проверить место и inode: `df -hT`, `df -i`
- 3 Проверить состояние RAID и LVM: `/proc/mdstat`, `pvs`, `vgs`, `lvs`
- 4 Проверить точки монтирования: `lsblk -f`, `mount`
- 5 Проверить журналы: `journalctl -xe`, `dmesg`
- 6 Выполнить действия по устранению
- 7 Повторно проверить систему

### ВАЖНО ПОМНИТЬ



- Всегда делайте резервные копии важных данных
- Проверяйте SMART и состояние RAID регулярно
- Не запускайте fsck на смонтированной файловой системе
- При отказе диска – действуйте быстро
- Следите за свободным местом (оставляйте 10–20%)
- Планируйте контроль: cron + отчёты

### ПОЛЕЗНЫЕ КОМАНДЫ НА ЗАМЕТКУ

|                                          |                  |                                      |                                   |
|------------------------------------------|------------------|--------------------------------------|-----------------------------------|
| <code>watch -n 1 df -hT</code>           | мониторинг места | <code>sudo vgchange -ay</code>       | активировать VG                   |
| <code>sudo dmesg -T   less</code>        | сообщения ядра   | <code>sudo lsdf +L1</code>           | найти удалённые, но занятые файлы |
| <code>sudo mdadm --monitor --scan</code> | мониторинг RAID  | <code>history   grep -i mount</code> | история монтирований              |

### КОГДА НУЖНА ПРОВЕРКА ФС

|                                |                            |
|--------------------------------|----------------------------|
| <code>fsck -n /dev/sdX1</code> | только проверка            |
| <code>fsck -y /dev/sdX1</code> | автоматическое исправление |



Выполнять только на НЕсмонтированной ФС!

## Отказ диска в базовой лаборатории проверяем просто

Команда: `cat /proc/mdstat`

Команда: `sudo mdadm --detail /dev/md0`

Команда: `lsblk`

На физическом сервере дополнительно важны Serial, WWN, Slot, BMC или RAID Controller

Аппаратная идентификация важна в production

В VirtualBox-лаборатории она не является основной частью



## Единый сценарий: шаги 1–3

Шаг 1: добавить два виртуальных диска `/dev/sdb` и `/dev/sdc`

Проверка: `lsblk`

Шаг 2: создать RAID1

Команда: `sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc`

Шаг 3: проверить RAID

Команда: `cat /proc/mdstat`

## Единый сценарий: шаги 4–7

Шаг 4: создать LVM

Команды: `sudo pvcreate /dev/md0; sudo vgcreate  
vg_data /dev/md0; sudo lvcreate -n lv_apps -L 10G  
vg_data`

Шаг 5: создать ext4

Команда: `sudo mkfs.ext4 /dev/vg_data/lv_apps`

Шаг 6: смонтировать /srv/apps

Команды: `sudo mkdir -p /srv/apps; sudo mount  
/dev/vg_data/lv_apps /srv/apps`



## Единый сценарий: шаги 8–10

Шаг 7: проверить mount

Команды: `lsblk -f; findmnt /srv/apps; df -hT /srv/apps`

Шаг 8: получить UUID и настроить /etc/fstab

Команда: `sudo blkid /dev/vg_data/lv_apps`

Шаг 9: проверить fstab командой `sudo mount -a`

Шаг 10: расширить LV и файловую систему

## Итоги лекции

Linux-хранилище нужно понимать слоями: Disk → RAID  
→ LVM → FS → Mount

RAID1 защищает от отказа одного диска, но не является backup

LVM дает гибкость через PV, VG и LV

После lvextend нужно расширить файловую систему  
или использовать -r

Snapshot полезен, но не заменяет backup

df, du и df -i отвечают на разные вопросы

UUID надежнее имен устройств в /etc/fstab

После изменения fstab сначала выполняют mount -a. и

## Контрольные вопросы

Чем диск отличается от файловой системы?

Что такое mount point?

Что означают PV, VG и LV?

Почему после lvextend иногда нужен resize2fs?

Для чего используется RAID1?

Почему RAID не является Backup?

Что показывают df, du и df -i?

Зачем использовать UUID в /etc/fstab?

Как проверить /etc/fstab без reboot?

Как определить состояние RAID1?